

Control law comparison

Six drives, eight control channels, low-damping frame (sdynpy_frame6x12_system.npz). Thirty-one runs, 2026-09-12 to 2026-09-20. Revision of the 2026-09-15 report.

Three things are new since the last revision. The fast optimal-diagonal law appears here *as a real run for the first time* — the two rows that carried its name in the twelve-run matrix did not execute its algorithm. A startup level cap and two live-FRF guards have been added to the laws and are documented here with the failures that motivated them. And every number in both tables has been recomputed under one method, so the historical and current results can be read against each other.

What a reader of the 2026-09-15 report needs to know Runs 05 and 06 are labelled optimal_diagonal_control_fast in that report and in the run files. They were the base class. The conclusion drawn from them — that the fast law matches the exact law closely — happened to be approximately right for the wrong reason, and the real measurement is in §3.

1. What the columns mean

rms dB	RMS of $10 \cdot \log_{10}(\text{measured} / \text{specification})$ over all eight control channels and all in-band lines (100–1000 Hz).
% >6 dB	Percentage of channel×line points outside ± 6 dB. <i>Note this is a ± 6 dB tolerance</i> ; the session notes quote a ± 3 dB figure, which is roughly double.
floor	Best achievable RMS error at this run's own FRF, from achievable_response.achievable_diagonal with restrict_rcond = 1e-3, every 5th in-band line, log-interpolated between solved lines. Not a constant — see §8.
vs floor	The run's error measured against its own floor rather than against the specification. This is the part attributable to the control law.
loop	What the law closes on. error: the measured response error. model: the identified plant model only — it re-solves against the live FRF but never looks at the measured response. none: one-shot synthesis, no loop at all. These labels changed on 2026-09-21 — see the note below.
level dB	Total in-band power, measured against specification.
drive	Mean in-band drive trace. Saved spectra are test-level normalised, so this is comparable across runs taken at different test levels.

2. The twelve-run matrix — historical baseline

Unchanged in substance from the 2026-09-15 report, recomputed here under the current method. Six laws, drive-coherence cap on and off.

The loop column has now changed twice, and this is the version to keep. The 2026-09-15 report called both optimal-diagonal laws feedback laws. That was wrong, and the correction made on 2026-09-17 relabelled them *open loop* — which over-shot, because it reads as though the plant model were frozen, when re-solving against a moving FRF is precisely what those laws do and what the 2026-09-19 work was about. Two different loops are at issue:

- model — the optimal-diagonal family. Closed on the identified plant model: the live FRF goes to `_refine_batch` every cycle, bins whose H moved are re-solved, and bins whose predicted error degraded against the *current* H are re-solved worst-first. Open on the response error: the objective is the *predicted* response $\text{diag}(H X H^H)$, and `last_response_cpsd` was accepted and never read. So a level-dependent FRF change is tracked, but error the FRF does not explain is invisible. §7 closes that half.
- error — `match_trace_pseudoinverse` (a trace ratio) and `match_diagonal_congruence` (a per-drive log error). These integrate the measured response error.
- none — `pseudoinverse_control` and `buzz_control`: one-shot syntheses, replayed unchanged.

No measured result in this table changed — only the label. But any conclusion in the 2026-09-15 report that leaned on “feedback laws do X ”, and any in the 2026-09-17 or 2026-09-20 versions that leaned on “these laws are open loop”, should be re-read with this in mind.

run	law	cap	loop	rms dB	% >6 dB	floor	vs floor	level dB	drive
run01	match_trace	0.95	error	8.22	27.5	3.88	7.16	+0.09	0.12
run02	match_trace	off	error	7.21	21.1	3.90	5.78	+0.05	6.02
run03	optimal_diagonal	0.95	model	5.62	11.2	3.64	3.99	+0.08	0.0747

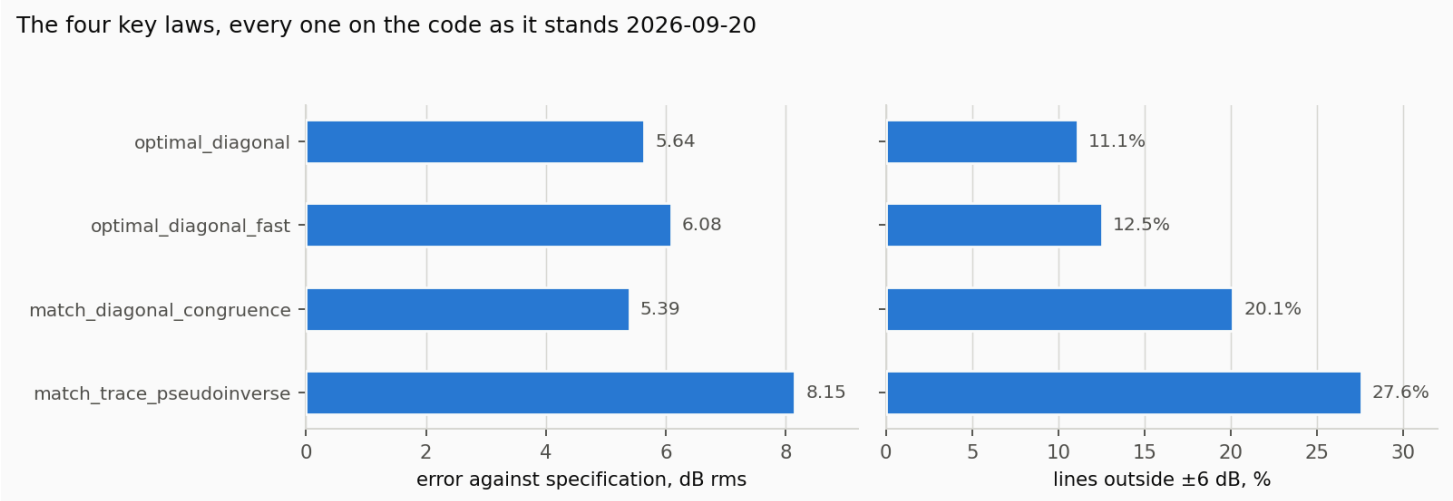
run	law	cap	loop	rms dB	% >6 dB	floor	vs floor	level dB	drive
run04	optimal_diagonal	off	model	5.62	11.2	3.61	3.99	+0.41	0.079
run05 NOT THE FAST LAW — SEE §2	optimal_diagonal_fast	0.95	model	5.64	10.9	3.75	3.99	+0.26	0.076
run06 NOT THE FAST LAW — SEE §2	optimal_diagonal_fast	off	model	5.66	10.7	3.61	3.99	+0.06	0.0756
run07	congruence	0.95	error	5.31	19.8	3.63	3.51	+2.85	0.0469
run08	congruence	off	error	5.23	18.7	3.57	3.48	+2.78	0.211
run09	pseudoinverse	0.95	none	10.47	42.1	3.92	9.55	+13.55	5.45
run10	pseudoinverse	off	none	7.01	17.8	3.92	5.48	-1.28	5.38
run11	buzz	0.95	none	7.90	30.2	3.82	6.75	+9.38	0.981
run12	buzz	off	none	5.91	13.6	3.85	4.20	-0.30	1.47

Why runs 05 and 06 are not fast-law data The fast law decides per call whether it is safe to use its unconstrained factored solve, by asking whether the FRF has moved since the last call. Until 2026-09-17 that test was bitwise equality. With “Update Transfer Function During Control” enabled — which is how all twelve runs were taken — Rattlesnake republishes an incrementally averaged FRF every cycle, so the array was never bit-identical, every bin fell through to the base class's SDP, and the law was byte-for-byte optimal_diagonal_control. It was only ever fast in the configuration it was least needed in. That the two rows nonetheless sit within 0.02 dB of runs 03 and 04 is not evidence about the fast law; it is the same law twice.

3. The four key laws on the current code

All four re-run between 2026-09-18 and 2026-09-20 on the code as it now stands, each with the drive-coherence cap at 0.95 and the FRF update on.

run	law	cap	loop	rms dB	% >6 dB	floor	vs floor	level dB	drive
run34	optimal_diagonal	0.95	model	5.64	11.1	3.71	4.01	+0.12	0.0745
run32	optimal_diagonal_fast	0.95	model	6.08	12.5	3.68	4.49	+0.01	0.018
run20	congruence	0.95	error	5.39	20.1	3.76	3.59	+3.04	0.0836
run33	match_trace	0.95	error	8.15	27.6	3.89	7.05	+0.17	0.187



Two measures of the same four runs. They are in different units, so they get two panels rather than two y-axes on one plot.

Two of the four are in good shape and two are not, for different reasons. optimal_diagonal_control is the accuracy benchmark and is stable across three code generations: 5.62 dB on run 03 (09-12), 5.68 on run 19 (09-18), 5.64 on run 34 (09-20), with drive within 0.4 % across all three. The fast law now runs its own algorithm and lands 0.44 dB behind it for roughly a quarter of the drive.

match_diagonal_congruence looks better than it is. Its 5.39 dB rms is second best on the board, but 20.1 % of its lines sit outside ±6 dB against optimal diagonal's 11.1 %, and it runs +3.04 dB hot. A low rms with that many lines out means the errors are *balanced*, not small. Forcing the overall level to 0 dB makes every measure worse — computed offline, rms 5.39 → 5.97 and the per-channel bias goes from 1.40 dB average high to 1.64 dB average low, because the +3 dB is what holds the two unreachable channels up. The level offset is not the problem; the per-line scatter is, and its cause is not yet known.

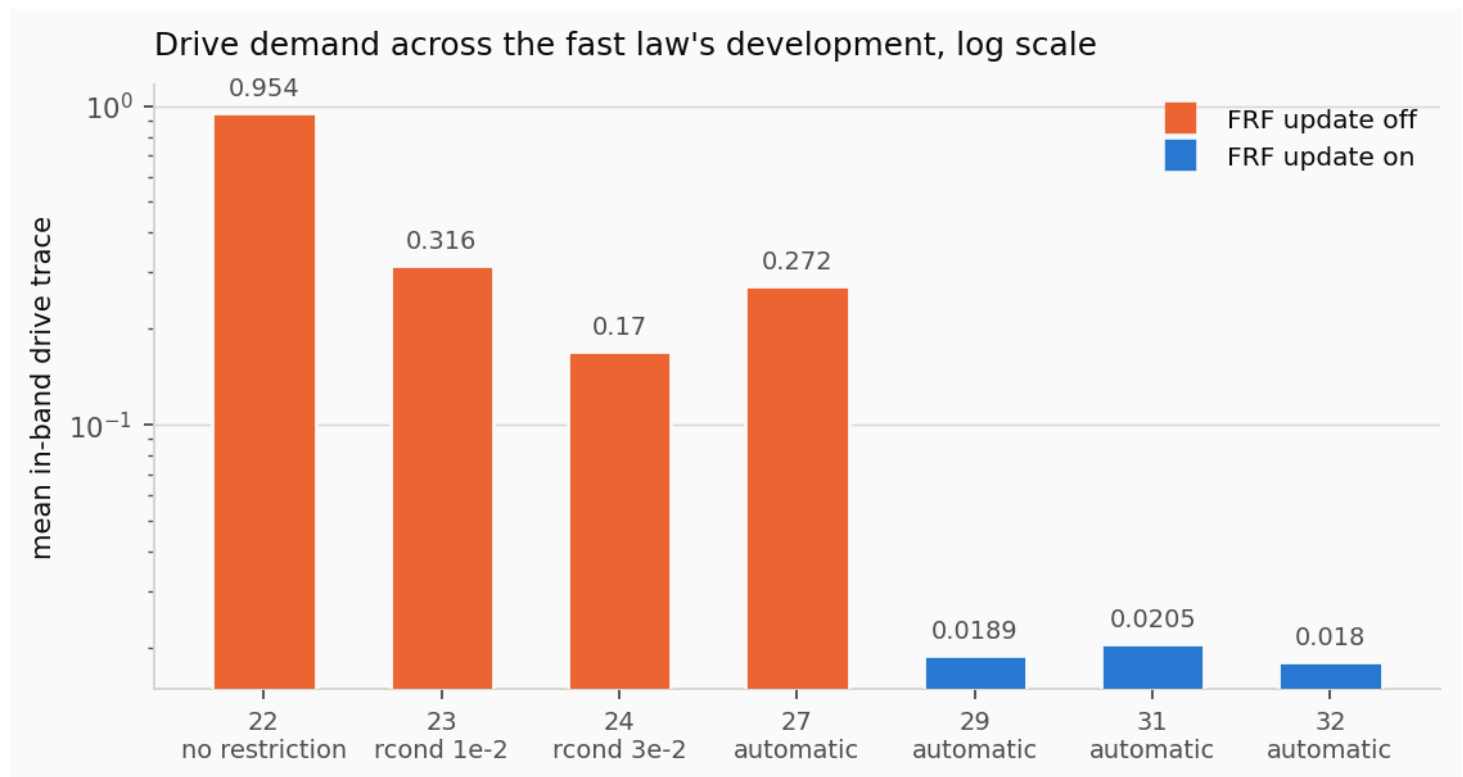
match_trace_pseudoinverse is the weakest, and now known to be weak for a structural reason rather than a stale-code one. Run 33 reproduced run 01 to within a tenth of a dB on every accuracy measure after eight days and three sets of changes to the file it lives in. Its per-channel mean bias runs [−2.08 −2.86 −2.57 −2.32 −0.86 −0.96] on the six reachable channels and −6.81/−7.48 on the two unreachable ones — every channel

below specification — while the overall level sits at +0.17 dB. That is only possible if a minority of lines sit far above. The law computes one real scalar per frequency line, `spec_trace/response_trace`, and scales the whole drive CPSD by it. The trace is therefore correct by construction and the distribution across the eight channels is whatever the pseudoinverse happens to give. Nothing in the law controls it.

4. The fast law, measured properly

Once the gate was fixed the factored solve turned out to have a failure mode of its own, and the sequence below is how it was found and bounded.

run	law	cap	loop	rms dB	% >6 dB	floor	vs floor	level dB	drive
run22	optimal_diagonal_fast	0.95	model	9.91	37.5	3.90	9.02	+14.29	0.954
run23	optimal_diagonal_fast	0.95	model	6.56	15.8	2.14	5.59	+3.55	0.316
run24	optimal_diagonal_fast	0.95	model	6.30	13.8	3.18	4.91	+0.75	0.17
run25	optimal_diagonal_fast	0.95	model	6.61	16.2	3.13	5.44	+1.17	0.178
run27	optimal_diagonal_fast	0.95	model	6.27	14.1	3.38	4.80	+0.55	0.272
run29	optimal_diagonal_fast	0.95	model	6.06	12.3	3.45	4.55	-0.10	0.0189
run31	optimal_diagonal_fast	0.95	model	6.04	12.2	3.65	4.44	-0.07	0.0205
run32	optimal_diagonal_fast	0.95	model	6.08	12.5	3.68	4.49	+0.01	0.018
run35	optimal_diagonal_fast	0.95	model	6.01	12.4	3.71	4.41	-0.02	0.0202



Mean in-band drive trace across the fast law's development. Log scale: the range spans a factor of 53.

Run 22 is the failure. With the factored path unconstrained in its default linear error domain, the solve commanded a drive whose predicted response was -0.39 dB and whose measured response was +14.29 dB, at 17.8 V. The solution was not wrong — it was *unrealizable*. Its accuracy lived in cancellation between drives that survives only if the drive is reproduced to a precision no rig delivers. Perturbing the commanded drive by 1 % offline predicts +16.69 dB, which brackets what the rig actually did.

drive_rcond is the fix. It confines the factored solve to the right-singular directions of H carrying $\sigma \geq rc \cdot \sigma_1$, so the objective cannot buy accuracy with drive down a near-null direction. Swept on the rig at 0, 1e-2, 3e-2 and 5e-2, the optimum was 3e-2. It is now expressed as a *multiple of the plant's own singular-value spread* — default 2.0 — because this frame's smallest structural direction sits at $\sigma_5/\sigma_1 = 0.0152$ and $2.0 \times 0.0152 = 0.030$ is the swept optimum. On a differently conditioned article the multiple tracks the plant where an absolute ratio would not. Runs 26 and 27 derived 0.0302 and 0.0303 from two fresh identifications without being told.

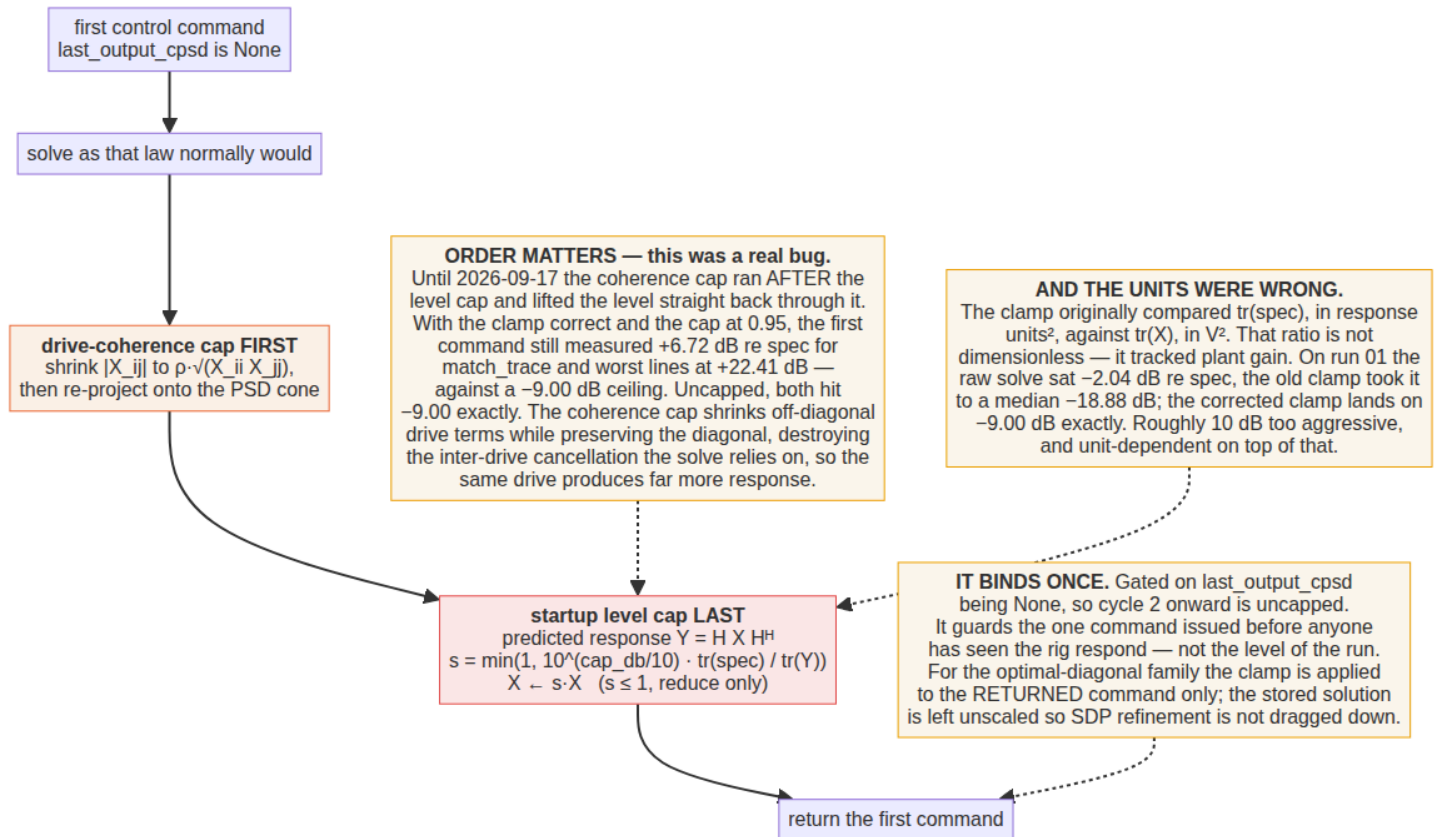
The FRF update turned out to matter more than the sweep. With the update off, the same commanded solution produced wildly different drive from run to run — 0.170, 0.538, 0.272 on runs 24, 26 and 27. With it on, four runs measured 0.0189, 0.0205, 0.0180 and 0.0202: a factor of 14 lower and reproducible to 7 %. Solving against a plant model that tracks the plant *as driven* removes the inter-drive cancellation the rig cannot reproduce. Runs 29, 31, 32 and 35 are four independent confirmations, and the acceptance bands for the second, third and fourth were

registered in the run sheet before those runs were taken. Run 35 — taken on 2026-09-21 with the response-error trim present in the code but off — landed at rms 6.01, level -0.02 and drive 0.0202, inside every band, which is what establishes that adding the trim changed nothing on the default path in the framework as well as in the harness.

The honest trade The SDP is still more accurate: 5.64 dB against 6.08, and 11.1 % of lines out against 12.5 %. The fast law's case is drive and time — about a quarter of the drive trace, and roughly 3.6× faster per bin on real data (not the 18–20× that synthetic-FRF benchmarks suggested).

5. The startup level cap

On the first control cycle there is no measured response yet, so a law falls back to a raw solve on whatever FRF the spectral processing has produced so far — which on the first frame is a single noisy unaveraged estimate, with no minimum-frames gate upstream. On real hardware this was observed to overshoot by several decades. The cap clamps the *predicted response trace* of that one command to `startup_test_level_cap_db` re the specification trace, default -9 dB.



The startup cap, and the two bugs found in it on 2026-09-17.

Two things about it were wrong until 2026-09-17 and are worth recording, because both were silent.

The units. The clamp originally compared the specification trace, in response units squared, against the drive trace, in volts squared. That ratio is not dimensionless — its value tracked plant gain and response-channel units. On run 01's identification the raw solve sat at -2.04 dB re specification and the clamp as written took it to a median -18.88 dB. The corrected clamp lands on -9.00 dB exactly.

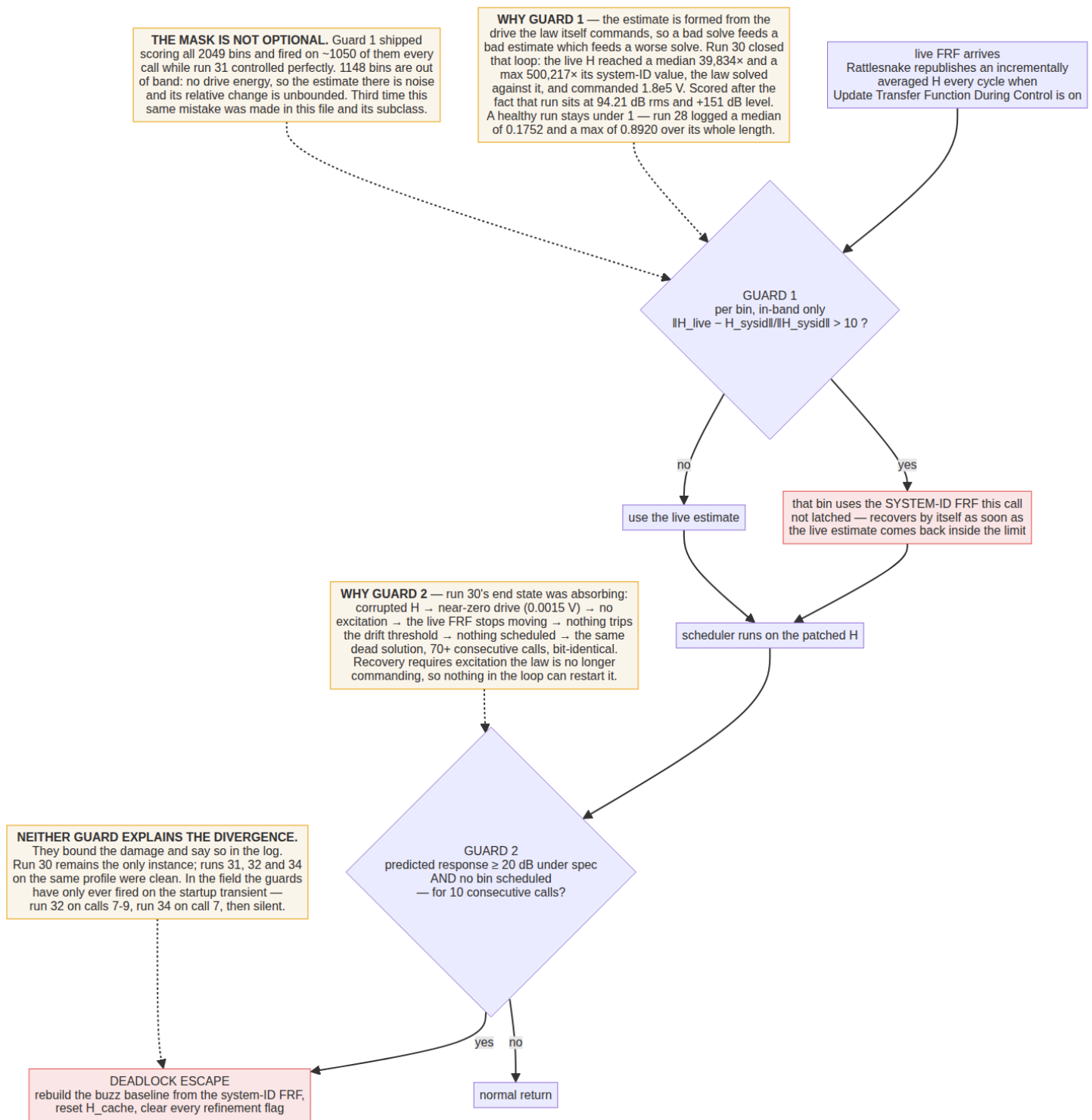
The order. Both callers ran the drive-coherence cap *after* the level clamp, which lifted the level straight back through it. With the clamp correct and the cap at 0.95, the first command still measured +6.72 dB re specification for `match_trace_pseudoinverse` and +22.41 dB on the worst single line, against a -9.00 dB ceiling. Uncapped, both hit -9.00 exactly. The coherence cap shrinks off-diagonal drive terms while preserving the diagonal, which destroys the inter-drive cancellation the pseudoinverse solve relies on, so the same drive produces far more response. The safety ceiling now goes last.

The cap binds on the first command only, in every law that has one. It guards the one command issued before anyone has seen the rig respond; it is not a level limit for the run.

6. The two live-FRF guards

Added 2026-09-19 to `optimal_diagonal_control`, and therefore inherited by `optimal_diagonal_control_fast` and `octave_band_switching_control`. They are *not* inherited by `match_diagonal_congruence` or `match_trace_pseudoinverse`, which are a standalone class and a plain function

respectively.



The two guards, and the run-30 failure each exists for.

Run 30 is why. With the FRF update on, Rattlesnake republishes an incrementally averaged estimate every cycle — formed from the drive the control law itself commands. A bad solve therefore feeds a bad estimate which feeds a worse solve. Run 30 closed that loop: the live FRF reached a median 39,834× and a maximum 500,217× its system-identification value, the law solved against it, and commanded 1.8×10^5 V. Scored after the fact, that run sits at **94.21 dB rms, 100 % of lines out, and +151 dB level** — and its own FRF reads numeric rank 2 rather than 5.

The demotion gate did not help, and that is the lesson. It fired correctly throughout run 30, demoting 69 % of calls to the coherence-capped SDP. Falling back to the safe solver cannot rescue a solve when the plant *model* is wrong, because both solvers get the same bad H. The gate protects against solver fragility; it was never a defence against estimator divergence, and those are different failures.

The end state was absorbing. Corrupted H produced a near-zero commanded drive (0.0015 V), which meant no excitation, so the live FRF stopped moving, so nothing tripped the drift threshold, so no bins were scheduled, so the law kept commanding the same dead solution — seventy consecutive calls with every scheduler field zero and a bit-identical self-prediction. Nothing in the loop could restart it, because restarting requires excitation the law was no longer commanding.

What the guards have and have not done Verified offline: on ordinary 3 % averaging jitter neither guard fires and the control output is bit-identical to the pre-guard code, so nothing in runs 01–29 is invalidated. In the field they have only ever fired on the startup transient — run 32 on calls 7, 8 and 9 covering 4, 20 and 3 in-band bins, and run 34 on call 7 covering 15. Two different laws, the same call numbers, the same handful of bins: that excursion is a property of the rig's startup, not of either law. **Run 30 remains the only divergence, and its trigger is unexplained.** Runs 31, 32 and 34 on the same profile were clean. The guards bound the damage; they do not explain it.

7. The response-error trim — closing the second loop

Added 2026-09-21, after the runs in this report. **Off by default and not yet run on the rig**, so nothing in the tables above reflects it.

The gap it closes is the one named in the note in §2. The optimal-diagonal family corrects whatever the FRF explains, because it re-solves against the live FRF every cycle. It cannot correct what the FRF does not explain — bias in the H1 estimate, extraneous input, drive clipping, a nonlinearity that never appears in a linear FRF. With no integral action on the measured response, such an offset simply stands, and the only sign of it is the law's own self-consistency diagnostic reading near 0 dB while Rattlesnake's Response Error panel reads high.

Two design choices carry the whole thing, and both were settled on paper before any code was written.

What is integrated. Not measured-versus-predicted, which is the obvious choice and is wrong: for a constant multiplicative plant bias b that ratio reads $10\log_{10}(b)$ for ever whatever the trim does, because shrinking the target shrinks prediction and measurement together. It is a mismatch readout, not an error, and an integrator driven by it runs to its clamp and stops there. The trim integrates measured-versus-specification instead — the quantity the test is judged on — which converges geometrically to the right correction.

What is excluded. Integrating measured-versus-specification alone would wind up at every bin the plant cannot reach, and this frame is rank 5 with six drives. So the trim is applied only where the solver reached what it aimed at, judged by the law's own predicted error against its own target, both cached from the cycle that issued the command. A shortfall the solver itself predicted is an achievability limit, not model error.

Run 35 measured how much of this there is to correct, and the answer changed the design. Reconstructing the law's own per-bin prediction from that run's saved FRF and drive, $\text{diag}(H D H^H)$, and comparing it with the measured response across all 7208 in-band channel-bins: **model mismatch median +0.008 dB, rms 0.060 dB, 90th percentile 0.054 dB.** There is essentially no model error on this plant. The FRF-update loop is already doing the whole job, and the residual 6.0 dB rms is the law's own optimisation limit — rank 5 against eight channels with a diagonal-only objective — not a gap between model and rig. This is the first time the self-consistency diagnostic has been checked against measurement rather than trusted, and it holds.

It also exposed a flaw. The gate originally reused `error_threshold_db` (1.0 dB). Swept against run 35's real data: at a 1.0 dB gate 46.6 % of the band is gated, 43.6 % of it outside the deadband on an error of 0.541 dB rms; at 0.5 dB, 26.3 % gated and 1.6 % outside; at 0.25 dB, 13.2 % and 0.3 %. The model mismatch stays at 0.041–0.047 dB rms throughout. At 1.0 dB the trim would have spent the entire run integrating the *solver's* own sub-threshold shortfall — thirteen times larger than the model error it exists to correct — and logging it as model correction. The gate is now its own parameter, `response_trim_gate_db`, default 0.5.

WHAT THIS FIXES.
Everything above this loop is driven by H. Error the FRF does not explain — H1 bias, extraneous input, drive clipping, a nonlinearity that never appears in a linear FRF — was invisible, because the law never looked at the measured response. There was no integral action, so the offset simply stood.

control() is called
Rattlesnake passes
last_response_cpsd —
the response to the
command issued last cycle

response_trim_gain > 0 ?
default 0.0

yes
frozen after a
test-level change?

yes

decrement, drop the
cached pairing, return
a frame spanning the ramp
is divided by the
wrong test level and is not
comparable

no

z =
diag(last_response_cpsd)
p = prediction cached for
the command
actually issued, AFTER the
startup cap
t_c = the target that
command was solved
against

in band, and z, p, t_c, S all
> 0 ?

no

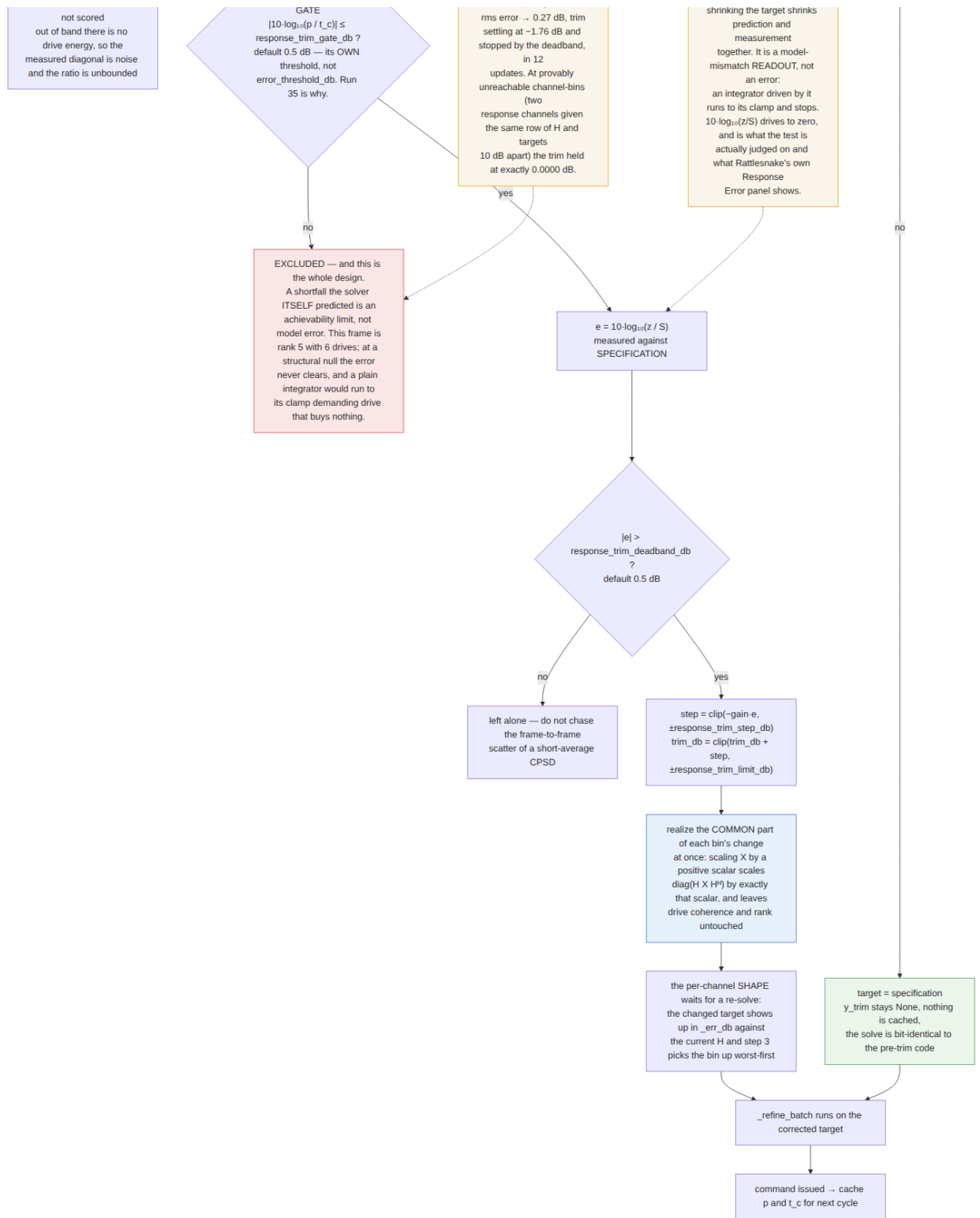
yes

SOLVER-AGREEMENT

WHY THE GATE HAS ITS OWN THRESHOLD. It first reused error_threshold_db = 1.0, on the reasoning that the scheduler's 'worth refining' bar and 'the solver reached what it aimed at' were the same question. Run 35 says they are not. On that run the model mismatch measured 0.060 dB rms over 7208 in-band channel-bins — but at a 1.0 dB gate, 43.6% of gated channel-bins cleared the deadband on an error of 0.541 dB rms. Thirteen to one: the trim would have integrated the SOLVER's own sub-threshold shortfall and logged it as model error. At 0.5 dB only 1.6% clear it. A real bias still opens the gate just as wide — the shortfall is computed entirely from H and X, so a plant that does not match H cannot narrow it.

MEASURED OFFLINE.
Against a plant 2.0 dB hotter than the model it was handed: on the channel-bins the solver reached: 2.09 dB

WHY NOT measured-vs-PREDICTED. For a constant multiplicative plant bias b, $10 \cdot \log_{10}(z/p)$ reads $10 \cdot \log_{10}(b)$ for ever no matter what the trim does —



The response-error trim. Which error signal is integrated, and the solver-agreement gate that keeps it from winding up at bins the plant cannot reach.

Bounded by a 0.5 dB per-cycle step, a ± 3 dB total authority clamp and a 0.5 dB deadband; scored in band only; and stood down for three cycles across a test-level change, because components/data_collector.py divides each acquired frame by the *current* test level and a frame spanning the ramp is normalised by the wrong number.

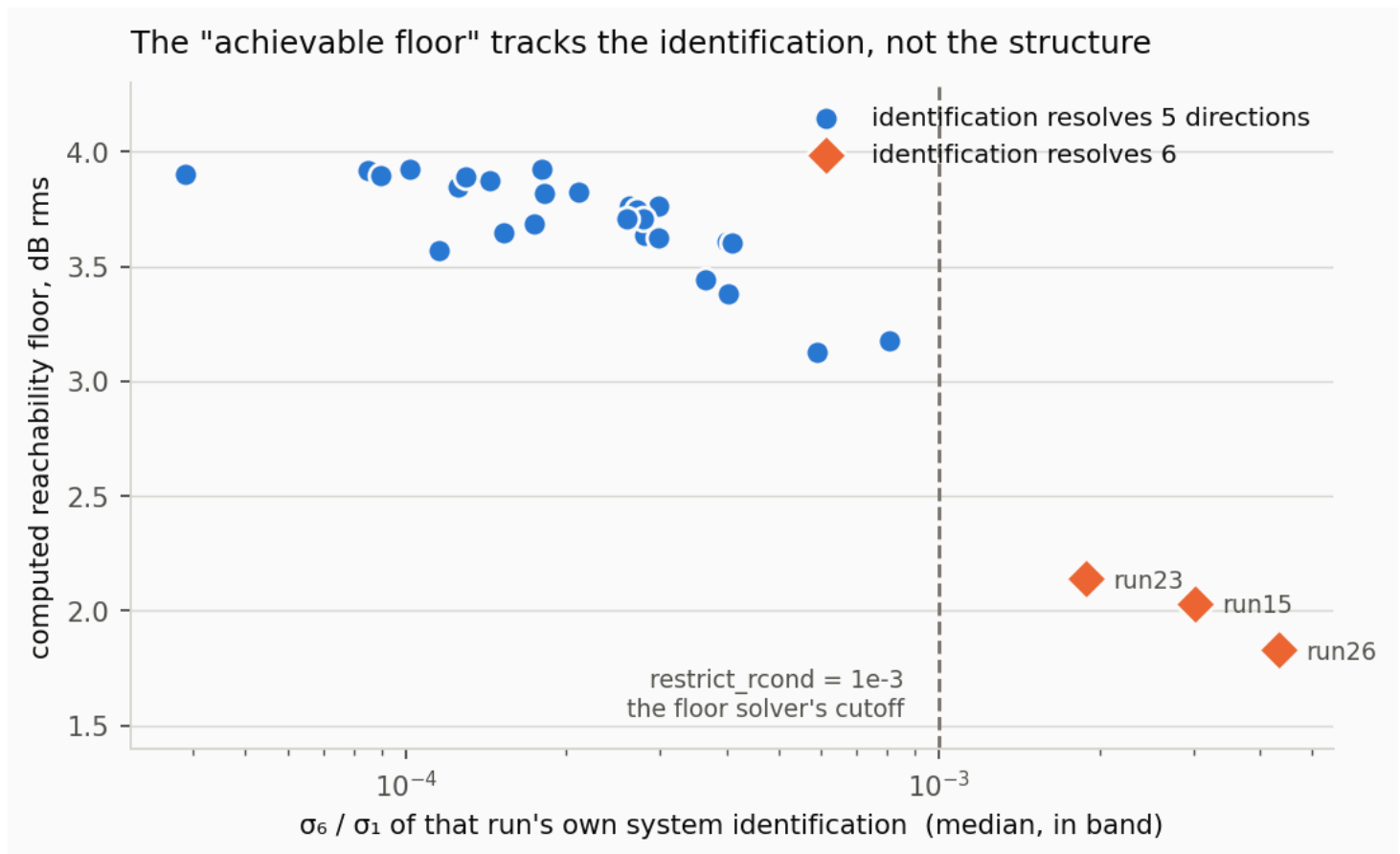
Offline results (verify_response_trim.py, four tests, all passing). With the gain at 0 the command is bit-identical to the committed code on real run-29 data, $\max |\text{new} - \text{committed}| = 0.000\text{e}+00$ — the test that had to hold, since otherwise every archived run would be in doubt. Against a plant 2.0 dB hotter than the model it was handed, on the channel-bins the solver reached: rms error 2.13 dB → 0.25 dB, trim settling at -1.76 dB and stopped by the deadband, in twelve updates. At provably unreachable channel-bins — two response channels given the same row of H and targets 10 dB apart, the solver's own shortfall a median 5.00 dB — the trim held at exactly 0.0000 dB.

Where this stands. Half-tested on the rig. Run 35 confirmed the default path — a full run with zero occurrences of either trim log line, so the trim is provably asleep inside the framework and not merely in a harness — and produced the mismatch measurement above. The trim has **not** been run with the gain on.

Run 35 also tells us what that run can and cannot show. With a model mismatch of 0.060 dB rms and an overall level already at -0.023 dB, there is almost nothing here to correct, so the pending run 36 is a does-no-harm test: a trim that stays nearly silent is the correct result. Nothing about this loop's gain or bandwidth should be claimed on the strength of it. The informative experiment needs a plant whose FRF genuinely moves with level — `build_nonlinear_frf_system.py`, `build_shifted_frf_system.py` and the untouched `sdynpy_frame6x12_system_higherz.npz` are all to hand, and that run would also open the five-untouched-plants item in §9.

8. A method finding: the floor is not a constant

Every “vs floor” number in this report is computed from the run's own FRF, and that turns out to matter more than expected.



Each point is one run. Run 30 is omitted: its FRF reads numeric rank 2 and its floor is not meaningful.

The floor solver keeps drive directions with $\sigma \geq 1e-3 \cdot \sigma_1$. This frame is structurally rank 5, and on most identifications the sixth singular value sits near $1e-4$ — comfortably below that cutoff, so the solver works with five directions and the floor lands between 3.13 and 3.92 dB. On runs 15, 23 and 26 the identification happened to lift σ_6 / σ_1 above $1e-3$; the solver got a sixth direction, and the floor collapsed to 1.83–2.14 dB.

So “error against the floor” is not comparable across runs whose identifications differ in quality. Run 26's apparently poor 5.61 dB against its floor and run 32's 4.49 are not measuring the same thing: run 26 is being judged against a floor that assumes a sixth drive direction the structure does not have. The sixth singular value is identification noise, not a property of the plant — which is exactly why `drive_rcond` finds the structural floor by gap detection rather than by taking $\sigma_{\min}$.

9. What is open

- **The response-error trim has never been run with the gain on.** Run 35 field-tested the default path and the four offline tests pass, but offline the plant bias was uniform, constant and exactly known, and on this plant the real mismatch is 0.060 dB rms — too small to exercise the loop. Runs 36 and 38 can show only that it does no harm. Until it has run against a plant whose FRF actually moves, the trim is a mechanism with a test behind it, not a result.

- **Whether polishing sub-threshold solver shortfall is worth doing deliberately.** The gate was tightened to 0.5 dB to stop the trim doing this by accident. At 1.0 dB it would push targets where the solver lands slightly low, which might squeeze out a few tenths of a dB — or might set the trim fighting the optimiser channel by channel. That is a separate experiment, and it should be run as one rather than arrived at through a default.
- **Run 30's trigger.** One instance, unexplained. Same profile, same test level, comparable identification, and three clean runs around it.
- **Congruence per-line scatter.** 20.1 % of lines outside ± 6 dB against optimal diagonal's 11.1 % at a lower rms. Cause unknown. This, not the level offset, is the real congruence problem.
- **Congruence repeatability.** Never run twice on identical code. This matters because the offset analysis in §3 was computed from a single run, and drive has been shown not to repeat on three other laws.
- **Plant coverage.** All 31 runs are on one plant, the low-damping frame. Five others sit unused in the results directory, and drive_rcond's automatic threshold has only ever been exercised on this one.
- **octave_band_switching_control** inherits both guards and has never been run in this campaign.

Generated 2026-09-20, revised 2026-09-21 (loop classification corrected; §7 added, then run 35 scored and the trim's gate given its own threshold), from report_data.json (31 runs, recomputed under one method by examples/sixdrive12resp/code/compute_report_data.py). Flow charts from Claude outputs/flowcharts/*.mmd. Companion document: *Control laws in the Rattlesnake MIMO random environment*, same date, which documents each law's algorithm in detail.